

Python pour la Data Science



Section 1 : Remise à niveau rapide sur Python



Section 2: Data Science avec Python



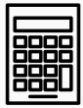
Section 3: Structure des données Panda



Section 4: Nettoyage des données



Section 5 : Visualisation des données sur Python



Section 6 : Analyse exploratoire des données



Section 7 : Séries temporelles



SERIES CHRONOLOGIQUES



Remise à niveau Python

Data Science avec Python

Python Pandas
Dataframes et séries

Visualisation de
données avec Python

Nettoyage des données

Analyse exploratoire des
données

Qu'est-ce qu'une série
chronologique ?

1

2

Mise en place Python

Qu'est-ce que Jupyter

3

4

Installation d'Anaconda
pour Windows OS

Installation d'Anaconda
pour Mac OS

5

6

Installation d'Anaconda
pour Ubuntu OS

Comment executer Python sur
Jupyter

7

8

Gérer les répertoires
dans Jupyter

Mise à l'echelle des
caractéristiques

9

10

Travailler avec différents
types de données

Variables

11

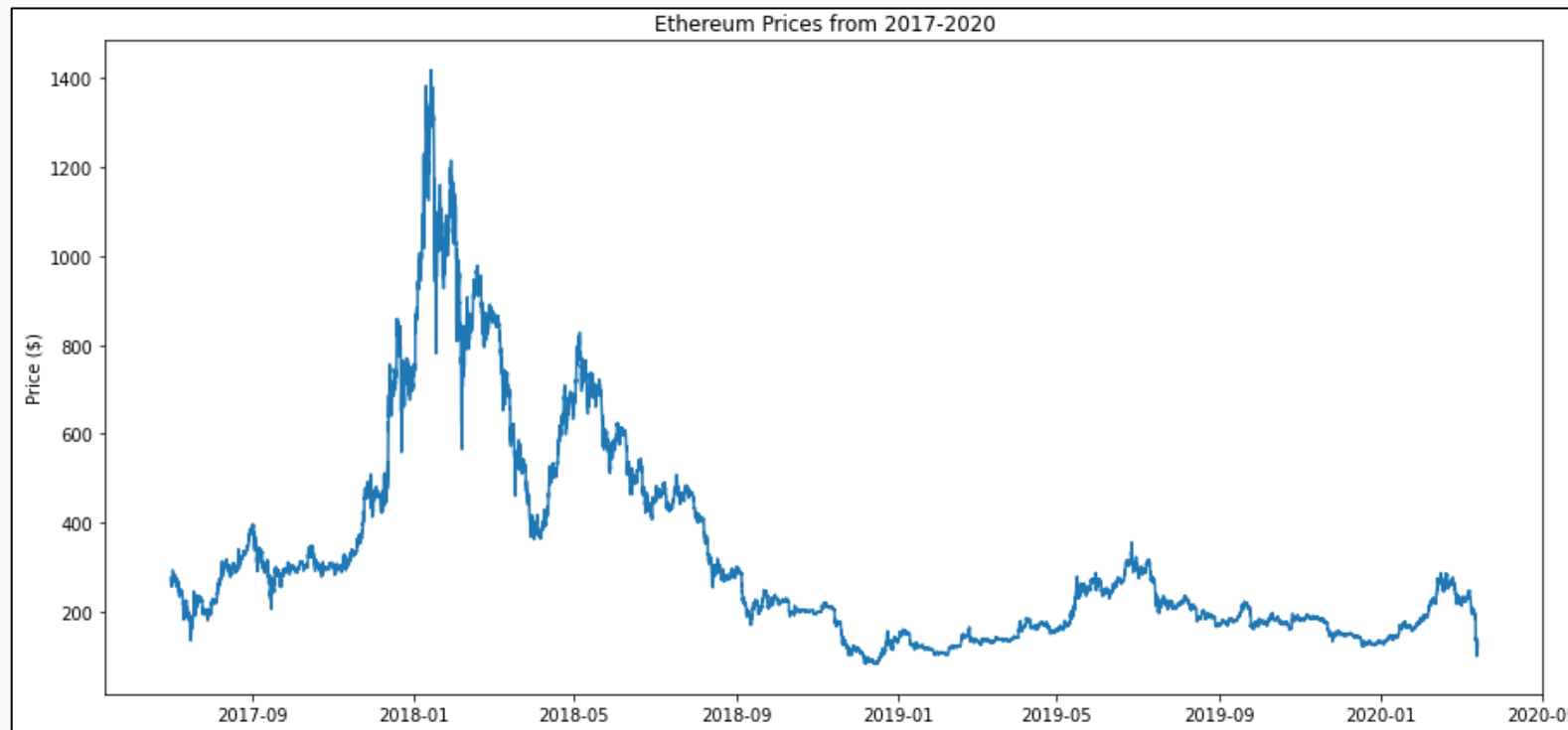
12

Opérateurs arithmétiques



Qu'est-ce qu'une série chronologique ?

- Une série chronologique est une séquence de données mesurées à intervalles réguliers.
- En analysant une série chronologique, nous pouvons déterminer la tendance dans les données et élaborer un modèle de prévision.
- Python fournit beaucoup de fonctionnalités pour analyser une série chronologique.





Remise à niveau Python

Data Science avec Python

Python Pandas
Dataframes et séries

Visualisation de
données avec Python

Nettoyage des données

Analyse exploratoire des
données

Introduction à Python

1

Qu'est-ce que Jupyter

2

**Bibliothèques Python pour l'analyse
des séries chronologiques**

3

Installation d'Anaconda
pour Mac OS

4

Installation d'Anaconda
pour Windows OS

5

Comment executer Python sur
Jupyter

6

Installation d'Anaconda
pour Ubuntu OS

7

Mise à l'echelle des
caractéristiques

8

Gérer les répertoires
dans Jupyter

9

Variables

10

Travailler avec différents
types de données

11

12

Opérateurs arithmétiques



Bibliothèques Python pour l'analyse des séries chronologiques

- Python a beaucoup de bibliothèques pour l'analyse de séries chronologiques.
- Certaines de ces bibliothèques sont utilisées dans le Machine Learning et sont donc hors du champ de ce cours.
- Nous utiliserons les bibliothèques suivantes pour l'analyse des séries chronologiques :
 - Pandas
 - NumPy
 - datetime
 - Matplotlib



Remise à niveau Python

Data Science avec Python

Python Pandas
Dataframes et séries

Visualisation de
données avec Python

Nettoyage des données

Analyse exploratoire des
données

Introduction à Python

1

2

Mise en place de Python

**Comment avoir une série
chronologique ?**

3

4

Installation d'Anaconda
pour Windows OS

Installation d'Anaconda
pour Mac OS

5

6

Installation d'Anaconda
pour Ubuntu OS

Comment executer Python sur
Jupyter

7

8

Gérer les répertoires
dans Jupyter

Mise à l'echelle des
caractéristiques

9

10

Travailler avec différents
types de données

Variables

11

12

Opérateurs arithmétiques



Comment obtenir des séries chronologiques?

- Comme nous l'avons déjà mentionné, une série chronologique n'est qu'un ensemble de valeurs mesurées à intervalles réguliers.
- En général, tout ensemble de données dont les valeurs sont mesurées au fil du temps peut être traité comme une série chronologique.
- Des exemples de tels ensembles de données pourraient comprendre, sans s'y limiter :
 - Données sur les stocks
 - Données météorologiques
 - Données sur l'évolution de la santé des patients
- En Python, le type de données de dates et d'heures dans une série chronologique est soit `datetime`, soit `Timestamp`.
- Dans ce cours, nous verrons comment télécharger une série chronologique et comment convertir un ensemble de données avec des valeurs mesurées au fil du temps en une série chronologique.



Remise à niveau Python

Data Science avec Python

Python Pandas
Dataframes et séries

Visualisation de
données avec Python

Nettoyage des données

Analyse exploratoire des
données

Introduction à Python

1

Mise en place de Python

2

Qu'est-ce que Jupyter

3

Installation d'Anaconda
pour Mac OS

4

**Obtenir des séries
chronologiques - yfinance**

5

Installation d'Anaconda
pour Ubuntu OS

6

Comment executer Python sur
Jupyter

7

Gérer les répertoires
dans Jupyter

8

Mise à l'echelle des
caractéristiques

9

Travailler avec différents
types de données

10

Variables

11

Opérateurs arithmétiques

12



Obtenir des données sur les stocks en utilisant yfinance (1/5)

- yfinance est une bibliothèque Python qui permet de télécharger des données de stocks de Yahoo Finance pour une période de temps spécifiée comme une série chronologique.
- Vous pouvez également télécharger un ensemble de données à partir de son site Web directement

<https://finance.yahoo.com/>





Obtenir des données sur les stocks en utilisant yfinance (2/5)

- Pour utiliser yfinance bibliothèque Python, nous devons d'abord l'installer via la commande suivante :
pip install yfinance
- Une fois la bibliothèque installée, nous pouvons l'importer dans le Jupyter Notebook.
- yfinance est généralement importé en tant que yf.

```
pip install yfinance

Collecting yfinanceNote: you may need to restart the kernel to use updated packages.

  Downloading yfinance-0.1.70-py2.py3-none-any.whl (26 kB)
Requirement already satisfied: lxml>=4.5.1 in c:\users\22100199\anaconda3\lib\site-packages (from yfinance) (4.6.3)
Requirement already satisfied: numpy>=1.15 in c:\users\22100199\anaconda3\lib\site-packages (from yfinance) (1.20.3)
Requirement already satisfied: requests>=2.26 in c:\users\22100199\anaconda3\lib\site-packages (from yfinance) (2.26.0)
Collecting multitasking>=0.0.7
  Downloading multitasking-0.0.10.tar.gz (8.2 kB)
Requirement already satisfied: pandas>=0.24.0 in c:\users\22100199\anaconda3\lib\site-packages (from yfinance) (1.3.4)
Requirement already satisfied: python-dateutil>=2.7.3 in c:\users\22100199\anaconda3\lib\site-packages (from pandas>=0.24.0->yfinance) (2.8.2)
Requirement already satisfied: pytz>=2017.3 in c:\users\22100199\anaconda3\lib\site-packages (from pandas>=0.24.0->yfinance) (2021.3)
Requirement already satisfied: six>=1.5 in c:\users\22100199\anaconda3\lib\site-packages (from python-dateutil>=2.7.3->pandas>=0.24.0->yfinance) (1.16.0)
Requirement already satisfied: idna<4,>=2.5 in c:\users\22100199\anaconda3\lib\site-packages (from requests>=2.26->yfinance) (3.2)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\22100199\anaconda3\lib\site-packages (from requests>=2.26->yfinance) (2021.10.8)
Requirement already satisfied: charset-normalizer~=2.0.0 in c:\users\22100199\anaconda3\lib\site-packages (from requests>=2.26->yfinance) (2.0.4)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in c:\users\22100199\anaconda3\lib\site-packages (from requests>=2.26->yfinance) (1.26.7)
Building wheels for collected packages: multitasking
  Building wheel for multitasking (setup.py): started
  Building wheel for multitasking (setup.py): finished with status 'done'
  Created wheel for multitasking: filename=multitasking-0.0.10-py3-none-any.whl size=8500 sha256=7d5ad1da18be483c99d9f99c43e6f458099dd09b5ec65cf632200300198faee6
  Stored in directory: c:\users\22100199\appdata\local\pip\cache\wheels\f2\b5\2c\59ba95dcf854e542944c75fe3da584e4e3833b319735a0546c
Successfully built multitasking
Installing collected packages: multitasking, yfinance
Successfully installed multitasking-0.0.10 yfinance-0.1.70

import yfinance as yf
```





Obtenir des données sur les stocks en utilisant yfinance (3/5)

- Après avoir importé la bibliothèque, nous pouvons télécharger les données de stock pour différentes entreprises en utilisant la méthode `.download()`.
- Nous passons une date de début et une date de fin au format « AAAA-MM-JJ ».
- S'il n'y a pas de données disponibles pour une certaine date, cette date serait sautée.



Obtenir des données sur les stocks en utilisant yfinance (4/5)

- Dans l'exemple donné, nous téléchargeons les données sur les actions d'Apple (abrégées AAPL dans Yahoo Finance) du 1er janvier 2022 au 31 janvier 2022.

```
df = yf.download('AAPL', start='2022-01-01', end='2022-01-31')  
df.head(10)
```

```
[*****100%*****] 1 of 1 completed
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2021-12-31	178.089996	179.229996	177.259995	177.570007	177.344055	64062300
2022-01-03	177.830002	182.880005	177.710007	182.009995	181.778397	104487900
2022-01-04	182.630005	182.940002	179.119995	179.699997	179.471344	99310400
2022-01-05	179.610001	180.169998	174.639999	174.919998	174.697418	94537600
2022-01-06	172.699997	175.300003	171.639999	172.000000	171.781143	96904000
2022-01-07	172.889999	174.139999	171.029999	172.169998	171.950928	86580100
2022-01-10	169.080002	172.500000	168.169998	172.190002	171.970901	106765600
2022-01-11	172.320007	175.179993	170.820007	175.080002	174.857224	76138300
2022-01-12	176.119995	177.179993	174.820007	175.529999	175.306641	74805200
2022-01-13	175.779999	176.619995	171.789993	172.190002	171.970901	84505800



Obtenir des données sur les stocks en utilisant yfinance (5/5)

- Comme vous pouvez le voir, les valeurs de la colonne d'index (Date) de l'ensemble de données de la diapositive précédente ont le type Timestamp, indiquant qu'il s'agit d'une série chronologique Python.

```
type(df.index[0])
```

```
pandas._libs.tslibs.timestamps.Timestamp
```



Quiz Time

1. Les données sur les stocks sur une période de intervalles réguliers sont-elles des séries chronologiques?

- Vrai
- Faux



Quiz Time

1. Les données sur les stocks sur une période de intervalles réguliers sont-elles des séries chronologiques?

- Vrai
- Faux



Remise à niveau Python

Data Science avec Python

Python Pandas
Dataframes et séries

Visualisation de
données avec Python

Nettoyage des données

Analyse exploratoire des
données

Introduction à Python

1

Mise en place de Python

2

Qu'est-ce que Jupyter

3

Installation d'Anaconda
pour Windows OS

4

**Conversion d'un ensemble de
données d'une série temporelle**

5

Installation d'Anaconda
pour Ubuntu OS

6

Comment executer Python sur
Jupyter

7

Gérer les répertoires
dans Jupyter

8

Mise à l'echelle des
caractéristiques

9

Travailler avec différents
types de données

10

Variables

11

Opérateurs arithmétiques

12



Conversion d'un ensemble de données d'une série chronologique (1/6)

- Les données de séries chronologiques sont généralement stockées sous forme de fichier csv ayant une colonne représentant la date ou l'heure ou les deux.
- Il peut être importé en Python en utilisant la fonction `.read_csv()` de la bibliothèque Pandas.
- Nous pouvons utiliser la fonction `to_datetime()` de la bibliothèque Pandas pour convertir les valeurs de la colonne de date de chaîne en Timestamp.



Conversion d'un ensemble de données d'une série chronologique (2/6)

- Considérez les séries chronologiques suivantes. Le type de chaque valeur de « date » (index) est une chaîne de caractères comme le montre la figure de droite.

df	
	value
date	
1991-07-01	3.526591
1991-08-01	3.180891
1991-09-01	3.252221
1991-10-01	3.611003
1991-11-01	3.565869
...	...
2008-02-01	21.654285
2008-03-01	18.264945
2008-04-01	23.107677
2008-05-01	22.912510
2008-06-01	19.431740
204 rows × 1 columns	

```
type(df.index[0])
```

```
str
```



Conversion d'un ensemble de données d'une série chronologique (3/6)

- Pour pouvoir utiliser la fonctionnalité de série chronologique disponible en Python, nous devons convertir les valeurs de « date » en horodatage.
- Cela peut être fait en utilisant la fonction `to_datetime()` de Pandas.

```
df.index = pd.to_datetime(df.index)
df
```

value	
date	
1991-07-01	3.526591
1991-08-01	3.180891
1991-09-01	3.252221
1991-10-01	3.611003
1991-11-01	3.565869
...	...
2008-02-01	21.654285
2008-03-01	18.264945
2008-04-01	23.107677
2008-05-01	22.912510
2008-06-01	19.431740
204 rows × 1 columns	

```
type(df.index[0])
```

```
pandas._libs.tslibs.timestamps.Timestamp
```



Conversion d'un ensemble de données d'une série chronologique (4/6)

- Dans l'exemple précédent, les données de la colonne « date » étaient dans un format que Pandas pouvait facilement convertir, mais ce n'est pas toujours le cas.
- Prenons l'exemple de la base de données suivante. Les valeurs de date sont des chaînes de caractères.

	Symbol	Open	High	Low	Close	Volume
Date						
2020-03-13 08-PM	ETHUSD	129.94	131.82	126.87	128.71	1940673.93
2020-03-13 07-PM	ETHUSD	119.51	132.02	117.10	129.94	7579741.09
2020-03-13 06-PM	ETHUSD	124.47	124.85	115.50	119.51	4898735.81
2020-03-13 05-PM	ETHUSD	124.08	127.42	121.63	124.47	2753450.92
2020-03-13 04-PM	ETHUSD	124.85	129.51	120.17	124.08	4461424.71
...	...	...	...	...	...	...
2017-07-01 03-PM	ETHUSD	265.74	272.74	265.00	272.57	1500282.55
2017-07-01 02-PM	ETHUSD	268.79	269.90	265.00	265.74	1702536.85
2017-07-01 01-PM	ETHUSD	274.83	274.93	265.00	268.79	3010787.99
2017-07-01 12-PM	ETHUSD	275.01	275.01	271.00	274.83	824362.87
2017-07-01 11-AM	ETHUSD	279.98	279.99	272.10	275.01	679358.87

23674 rows × 6 columns



Conversion d'un ensemble de données d'une série chronologique (5/6)

- La conversion des valeurs de « Date » (index) donne une erreur « ParserError ».

```
df.index = pd.to_datetime(df.index)
df
```

```
-----
TypeError                                Traceback (most recent call last)
File ~\Anaconda3\lib\site-packages\pandas\core\arrays\datetime.py:2187, in objects_to_datetime64ns(data, dayfirst, yearfirst,
utc, errors, require_iso8601, allow_object, allow_mixed)
    2186 try:
-> 2187     values, tz_parsed = conversion.datetime_to_datetime64(data.ravel("K"))
    2188     # If tzaware, these values represent unix timestamps, so we
    2189     # return them as i8 to distinguish from wall times

File ~\Anaconda3\lib\site-packages\pandas\_libs\tslibs\conversion.pyx:359, in pandas._libs.tslibs.conversion.datetime_to_dateti
me64()

TypeError: Unrecognized value type: <class 'str'>

During handling of the above exception, another exception occurred:
```

```
ParserError                                Traceback (most recent call last)
Input In [32], in <module>
----> 1 df.index = pd.to_datetime(df.index)
      2 df
```



Conversion d'un ensemble de données d'une série chronologique (6/6)

- L'erreur survient parce que Pandas ne connaît pas le format des valeurs de chaîne.
- Nous pouvons utiliser le paramètre « format » pour résoudre ce problème.
- Vous pouvez trouver une liste de tous les codes de format à :

<https://docs.python.org/3/library/datetime.html#strptime-and-strftime-behavior>

```
df.index = pd.to_datetime(df.index, format='%Y-%m-%d %I-%p')
print(type(df.index[0]))
df
```

```
<class 'pandas._libs.tslibs.timestamps.Timestamp'>
```

	Symbol	Open	High	Low	Close	Volume
Date						
2020-03-13 20:00:00	ETHUSD	129.94	131.82	126.87	128.71	1940673.93
2020-03-13 19:00:00	ETHUSD	119.51	132.02	117.10	129.94	7579741.09
2020-03-13 18:00:00	ETHUSD	124.47	124.85	115.50	119.51	4898735.81
2020-03-13 17:00:00	ETHUSD	124.08	127.42	121.63	124.47	2753450.92
2020-03-13 16:00:00	ETHUSD	124.85	129.51	120.17	124.08	4461424.71
...	...	...	...	...	...	...
2017-07-01 15:00:00	ETHUSD	265.74	272.74	265.00	272.57	1500282.55
2017-07-01 14:00:00	ETHUSD	268.79	269.90	265.00	265.74	1702536.85
2017-07-01 13:00:00	ETHUSD	274.83	274.93	265.00	268.79	3010787.99
2017-07-01 12:00:00	ETHUSD	275.01	275.01	271.00	274.83	824362.87
2017-07-01 11:00:00	ETHUSD	279.98	279.99	272.10	275.01	679358.87

23674 rows × 6 columns



Remise à niveau Python

Data Science avec Python

Python Pandas
Dataframes et séries

Visualisation de
données avec Python

Nettoyage des données

Analyse exploratoire des
données

Introduction à Python

1

Mise en place de Python

2

Qu'est-ce que Jupyter

3

Installation d'Anaconda
pour Windows OS

4

Installation d'Anaconda
pour Mac OS

5

**Analyse statistique
d'une série temporelle**

6

Comment executer Python sur
Jupyter

7

Gérer les répertoires
dans Jupyter

8

Mise à l'echelle des
caractéristiques

9

Travailler avec différents
types de données

10

Variables

11

Opérateurs arithmétiques

12



Travailler avec des séries chronologiques (1/7)

- Dans cette section, nous apprendrons à utiliser la fonctionnalité de série chronologique Pandas.
- Nous utiliserons les séries chronologiques suivantes contenant les données de stock d'Apple du 1er janvier 2022 au 4 février 2022.

```
df = yf.download('AAPL', start='2022-01-01', end='2022-02-04')  
df.head(10)
```

[*****100%*****] 1 of 1 completed

	Open	High	Low	Close	Adj Close	Volume
Date						
2021-12-31	178.089996	179.229996	177.259995	177.570007	177.344055	64062300
2022-01-03	177.830002	182.880005	177.710007	182.009995	181.778397	104487900
2022-01-04	182.630005	182.940002	179.119995	179.699997	179.471344	99310400
2022-01-05	179.610001	180.169998	174.639999	174.919998	174.697418	94537600
2022-01-06	172.699997	175.300003	171.639999	172.000000	171.781143	96904000
2022-01-07	172.889999	174.139999	171.029999	172.169998	171.950928	86580100
2022-01-10	169.080002	172.500000	168.169998	172.190002	171.970901	106765600
2022-01-11	172.320007	175.179993	170.820007	175.080002	174.857224	76138300
2022-01-12	176.119995	177.179993	174.820007	175.529999	175.306641	74805200
2022-01-13	175.779999	176.619995	171.789993	172.190002	171.970901	84505800



Travailler avec des séries chronologiques (2/7)

.day_name()

- La première chose que vous avez peut-être remarquée, c'est qu'il n'y a pas de données disponibles pour les 1er et 2 janvier 2022.
- Voyons si ces deux jours étaient ouvrables ou non.
- Nous utiliserons la fonction .day_name() pour savoir quel jour était le 1 janvier.
- Rappelez-vous que cette fonction ne peut être appliquée que sur une valeur de datetime ou Timestamp.
- Le 1er janvier 2022 était un samedi et donc le 2 janvier 2022 était un dimanche. Pas d'affaires !

```
jan1 = pd.to_datetime('2022, 1, 1')  
jan1.day_name()
```

```
'Saturday'
```



Travailler avec des séries chronologiques (3/7)

Date minimale/maximale

- Parfois, nos données de séries chronologiques peuvent ne pas être en ordre.
- Dans un tel cas, nous serions intéressés par ce qui est le maximum et le temps minimum (ou horodatage) dans l'ensemble de données.
- Cela peut être fait en utilisant les fonctions `.max()` et `.min()`.

```
df.index.max()
```

```
Timestamp('2022-02-03 00:00:00')
```

```
df.index.min()
```

```
Timestamp('2021-12-31 00:00:00')
```



Travailler avec des séries chronologiques (4/7)

Gamme datetime

- Voyons maintenant quelle est la plage de dates dans notre ensemble de données.
- Ceci est aussi simple que de soustraire la valeur minimale de datetime (Timestamp) de sa valeur maximale.
- Notez que cela ne fonctionnera que si vous avez des valeurs datetime ou Timestamp dans votre jeu de données et non des chaînes.

```
timePeriod = df.index.max() - df.index.min()  
timePeriod  
  
Timedelta('34 days 00:00:00')
```



Travailler avec des séries chronologiques (5/7)

Indexation basée sur le temps

- Pandas fournit une indexation temporelle pour les séries chronologiques, c'est-à-dire l'accès aux données en utilisant datetime.
- Nous pouvons sélectionner des données sur des dates et des heures spécifiques en utilisant `.loc`.
- Dans l'exemple donné, nous obtenons les données le 7 janvier 2022.

```
df.loc['2022-01-07']
```

Open	1.728900e+02
High	1.741400e+02
Low	1.710300e+02
Close	1.721700e+02
Adj Close	1.719509e+02
Volume	8.658010e+07
Name: 2022-01-07 00:00:00, dtype: float64	



Travailler avec des séries chronologiques (6/7)

Indexation basée sur le temps

- Choisissons plusieurs dates à l'aide du tranchage avec `.loc`.

```
df.loc['2022-01-07':'2022-01-14']
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2022-01-07	172.889999	174.139999	171.029999	172.169998	171.950928	86580100
2022-01-10	169.080002	172.500000	168.169998	172.190002	171.970901	106765600
2022-01-11	172.320007	175.179993	170.820007	175.080002	174.857224	76138300
2022-01-12	176.119995	177.179993	174.820007	175.529999	175.306641	74805200
2022-01-13	175.779999	176.619995	171.789993	172.190002	171.970901	84505800
2022-01-14	171.339996	173.779999	171.089996	173.070007	172.849792	80355000



Travailler avec des séries chronologiques (7/7)

Indexation fondée sur le temps

- Nous pouvons également faire correspondre partiellement un datetime en utilisant `.loc`.
- Dans l'exemple donné, nous obtenons les données des séries chronologiques pour le mois de février seulement.
- Nous passons la chaîne « 2022-02 » pour obtenir les données de février.

```
df.loc['2022-02']
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2022-02-01	174.009995	174.839996	172.309998	174.610001	174.387817	86213900
2022-02-02	174.750000	175.880005	173.330002	175.839996	175.616257	84914300
2022-02-03	174.479996	176.240005	172.119995	172.899994	172.679993	89418100



Quiz Time

1. Considérez le dataframe donné appelé df. Quelle valeur la commande suivante va-t-elle retourner ?

`df.loc['2008-02-01']`

- a) 29.665356
- b) 21.654285
- c) 18.264945
- d) Erreur

	value
date	
2008-01-01	29.665356
2008-02-01	21.654285
2008-03-01	18.264945
2008-04-01	23.107677
2008-05-01	22.912510
2008-06-01	19.431740



Quiz Time

1. Considérez le dataframe donné appelé df. Quelle valeur la commande suivante va-t-elle retourner ?

`df.loc['2008-02-01']`

- a) 29.665356
- b) 21.654285
- c) 18.264945
- d) Erreur

	value
date	
2008-01-01	29.665356
2008-02-01	21.654285
2008-03-01	18.264945
2008-04-01	23.107677
2008-05-01	22.912510
2008-06-01	19.431740



Remise à niveau Python

Data Science avec Python

Python Pandas
Dataframes et séries

Visualisation de
données avec Python

Nettoyage des données

Analyse exploratoire des
données

Introduction à Python

1

Mise en place de Python

2

Qu'est-ce que Jupyter

3

Installation d'Anaconda
pour Windows OS

4

Installation d'Anaconda
pour Mac OS

5

Installation d'Anaconda
pour Ubuntu OS

6

**Visualisation d'une série
temporelle**

7

Gérer les répertoires
dans Jupyter

8

Mise à l'échelle des
caractéristiques

9

Travailler avec différents
types de données

10

Variables

11

Opérateurs arithmétiques

12



Visualisation d'une série chronologique (1/3)

- En utilisant Matplotlib et seaborn, nous pouvons visualiser une série chronologique.
- Voyons comment les cours boursiers d'Apple ont varié au cours des 5 dernières années (2017-2021).
- Nous commencerons par télécharger les données sur les stocks de 2017 à 2021.

```
df = yf.download('AAPL', start='2017-01-01', end='2021-12-31')
df
```

[*****100%*****] 1 of 1 completed

Date	Open	High	Low	Close	Adj Close	Volume
2017-01-03	28.950001	29.082500	28.690001	29.037500	27.297691	115127600
2017-01-04	28.962500	29.127501	28.937500	29.004999	27.267141	84472400
2017-01-05	28.980000	29.215000	28.952499	29.152500	27.405800	88774400
2017-01-06	29.195000	29.540001	29.117500	29.477501	27.711329	127007600
2017-01-09	29.487499	29.857500	29.485001	29.747499	27.965153	134247600
...	...	...	...	...	...	...
2021-12-23	175.850006	176.850006	175.270004	176.279999	176.055695	68356600
2021-12-27	177.089996	180.419998	177.070007	180.330002	180.100540	74919600
2021-12-28	180.160004	181.330002	178.529999	179.289993	179.061859	79144300
2021-12-29	179.330002	180.630005	178.139999	179.380005	179.151749	62348900
2021-12-30	179.470001	180.570007	178.089996	178.199997	177.973251	59773000

1258 rows × 6 columns





Visualisation d'une série chronologique (2/3)

- Ensuite, utilisons la fonction `.plot()` de `matplotlib.pyplot` pour tracer la série temporelle 'Adj Close'.
- On peut voir que les cours boursiers d'Apple ont considérablement augmenté depuis 2017.

```
plt.figure(figsize=(15, 5))  
plt.title('Apple Stock Prices for 5 years')  
plt.xlabel('Year')  
plt.ylabel('Price')
```

```
plt.plot(df['Adj Close'])
```

```
[<matplotlib.lines.Line2D at 0x2639bfe75b0>]
```





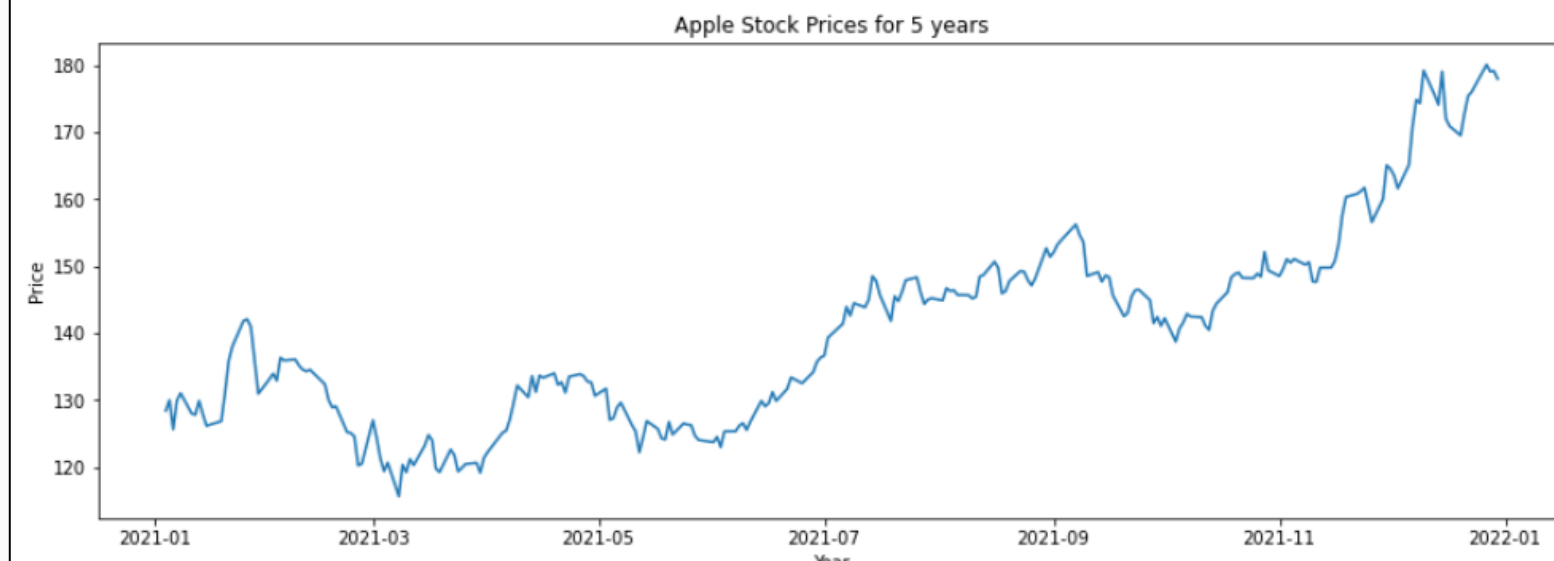
Visualisation d'une série chronologique (3/3)

- Nous pouvons également voir comment les cours des actions d'Apple ont varié au cours de 2021 seulement.
- Nous utilisons l'indexation temporelle pour obtenir l'année 2021 à partir de l'ensemble de données.
- Nous traçons ensuite la série chronologique « Adj Close ».

```
plt.figure(figsize=(15, 5))  
plt.title('Apple Stock Prices for 5 years')  
plt.xlabel('Year')  
plt.ylabel('Price')
```

```
plt.plot(df.loc['2021']['Adj Close'])
```

```
[<matplotlib.lines.Line2D at 0x2639c3010d0>]
```





Merci beaucoup !